

Shift-register patterns

A shift register is a chain of flip-flops where each stage passes its value to the next one on every clock edge

How shifting works

- On each rising edge, each flip-flop captures the output of the flip-flop to its left
- The leftmost stage captures the serial input
- After N clock cycles an N-bit shift register has moved each bit N positions to the right
- For parallel output, you read all stages at once
- For parallel load
- The ring version connects the last stage back to the first

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Shift-register lab

INTERACTIVE TOOL

Shift-register lab

Step a 4-bit chain through **SISO**, **SIPO**, **PISO**, and **PIPO** — serial vs parallel ports on the same flip-flop row.

Starter example: SISO with si=1, empty chain — step the clock to walk bits toward serial out.

Load starter example

Challenges (1/22)

Quiz: **SISO**. Serial-in serial-out acronym? Answer: SISO

Answer

Show hint

Check

Next

Idle

quiz-siso

quiz-sipo

quiz-piso

quiz-pipo

starter

preset-siso

preset-sipo

preset-piso

preset-pipo

toggle-si

step

shift-in

demo

parallel-load

toggle-load

pipo-load

clear

explain

mode-sipo

code-siso

code-piso

full-path

Core ideas

SISO

One bit in, one bit out — delay line / serializer core.

SIPO

Serial gather, then read the parallel word.

PISO

Load a word, then clock bits out serially.

PIPO

Ordinary parallel register (load / hold).

Shift-register lab starter

learn_verilog — shift register lab

Real Verilog practice

wsl - module15-shift-register-lab/examples/navigation

```
# Track A - real Unix
# real Verilog session (Track A)

$ cat examples/shift_demo/shift_demo.v
// shift_demo.v - SIPO and PISO shift registers
module sipo8(
    input    clk,
    input    rst,
    input    si,
    output reg [7:0] q
);
    always @(posedge clk) begin
        if (rst) q ≤ 8'd0;
        else    q ≤ {q[6:0], si};
    end
endmodule

module piso8(
    input    clk,
    input    rst,
    input    load,
    input    [7:0] d,
    output reg so,
    output reg [7:0] shreg
);
    always @(posedge clk) begin
        if (rst)    shreg ≤ 8'd0;
        else if (load) shreg ≤ d;
        else       shreg ≤ {1'b0, shreg[7:1]};
    end
    always @(posedge clk)
        so ≤ shreg[0];
endmodule

$ iverilog -t null examples/shift_demo/shift_demo.v
(syntax check passed)
```

Real Verilog - shift register demo

Pitfalls to watch

- Do not use blocking assignment inside a clocked shift register
- Do not forget to account for the latency
- When connecting stages manually instead of using a concatenation
- And when using a ring, verify the feedback path or you may lose bits silently

Your turn

- Complete the checklist for at least one track, ideally both
- In the browser lab
- In real Verilog
- When you finish, take the short quiz